

App Development Project Report

Placement Portal Application — V2 (Modern Application Development II)

1. Student Details

Name - Sreevatsan T

Roll Number - 21F3002380

Email - 21f3002380@ds.study.iitm.ac.in

About Me – I am a Software Engineer at FIS, who is passionate about data science and programming, particularly in data analytics and data engineering. I like building projects that help me learn more about development in general.

2. Project Details

Project Title: Placement Portal Application — V2

Problem Statement:

Institutes rely on spreadsheets, email, and manual coordination to manage campus recruitment — company approvals, placement drives, student applications, and placement tracking. This project builds a web application allowing an Admin (Institute Placement Cell), Companies, and Students to interact with the system based on their roles, replacing that manual process with structured, role-based workflows.

Approach:

The application was built as a decoupled Flask JSON API paired with a Vue 3 single-page frontend (loaded via CDN, no build step). Authentication uses JWT bearer tokens with role-based access control across a unified User model (admin/company/student). Admins pre-exist in the system; companies register and wait for admin approval before they can log in; students self-register immediately. Redis provides caching for search/listing endpoints with active invalidation on writes. Celery, backed by Redis as the broker, runs three background jobs: daily interview reminders, a monthly placement report emailed to the admin, and a user-triggered asynchronous CSV export — plus an additional job that auto-generates and emails an offer letter/placement confirmation whenever a company marks an applicant Selected or Placed. All emails are routed through a local SMTP server (MailHog) for testing/demo purposes.

3. AI/LLM Declaration

I used Claude Haiku to assist in writing SQLAlchemy model definitions, creating API documentation samples, and improving variable naming consistency. The extent of AI/LLM usage is around 10–15%, limited to code suggestions and documentation formatting. All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend — pure JSON API (no server-rendered HTML)
Vue 3 (CDN)	Frontend UI — single-page app, no build step
Vue Router	Client-side routing with role-based navigation guards
SQLAlchemy / Flask-SQLAlchemy	ORM for the SQLite database
SQLite	Database (created programmatically via <code>db.create_all()</code>)
PyJWT	JWT token creation and verification for authentication
Flask-Bcrypt	Password hashing
Bootstrap 5	Frontend styling and responsive layout
Chart.js	Analytics — application funnel, placement trend, skills demand
Redis	API response caching (search/listing endpoints)
Celery + Redis	Background jobs — interview reminders, monthly report, async CSV export, offer-letter generation
SMTP (MailHog for local testing)	Email delivery for reminders, reports, and offer letters

5. Database Schema / ER Diagram

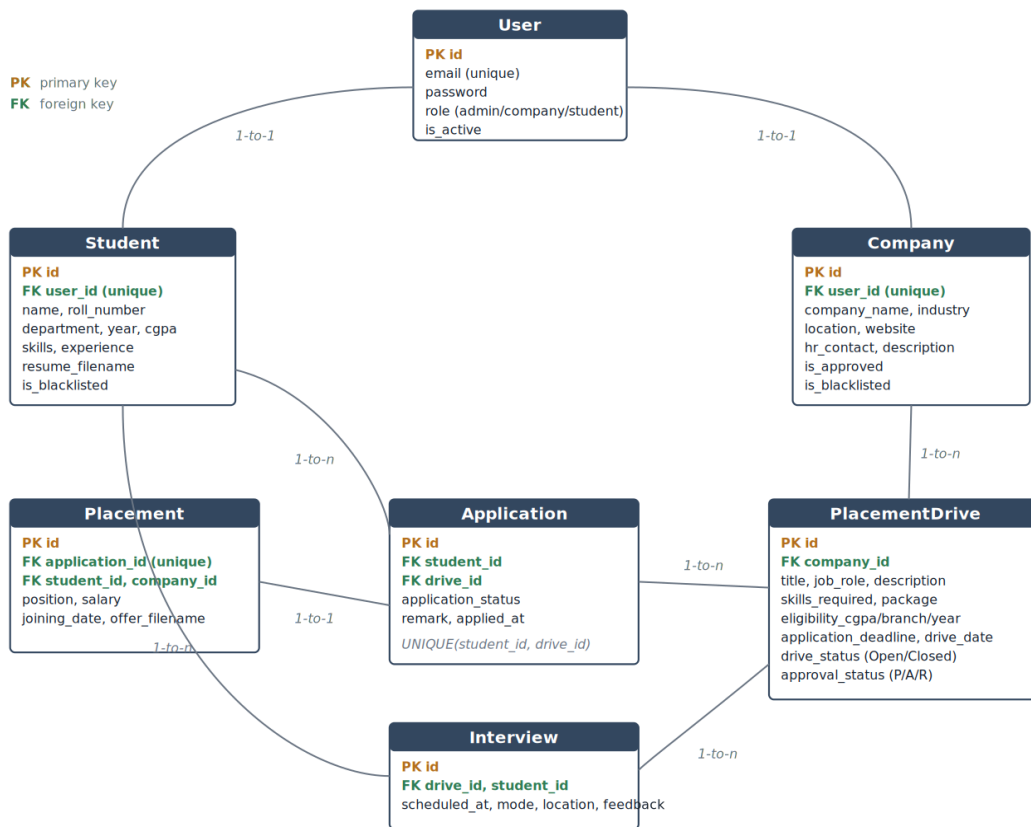
Tables:

- User — unified account table (id, email, password, role, is_active)
- Student — profile 1:1 with User (name, roll_number, department, year, cgpa, skills, resume_filename, is_blacklisted)
- Company — profile 1:1 with User (company_name, industry, location, hr_contact, website, is_approved, is_blacklisted)
- PlacementDrive — 1:n from Company (title, job_role, package, eligibility criteria, application_deadline, approval_status, drive_status)
- Application — 1:n from Student and from PlacementDrive; `UNIQUE(student_id, drive_id)` prevents duplicate applications
- Interview — 1:n from PlacementDrive and Student (scheduled_at, mode, location, feedback)
- Placement — 1:1 with Application; 1:n from Student and Company (position, salary, joining_date, offer_filename)

Relationships:

- User 1:1 Student, User 1:1 Company
- Company 1:n PlacementDrive
- PlacementDrive 1:n Application, PlacementDrive 1:n Interview
- Student 1:n Application, Student 1:n Interview
- Application 1:1 Placement

Placement Portal V2 — ER Diagram



6. API Resource Endpoints

Representative sample below (41 endpoints total across Auth / Admin / Company / Student / Public). Full definitions in the accompanying api.yaml (OpenAPI 3.0 format).

Endpoint	Method	Description
/api/auth/register	POST	Register a student or company account
/api/auth/login	POST	Authenticate and receive a JWT
/api/admin/dashboard	GET	Admin counts, pending approvals, recent applications
/api/admin/companies	GET	Search/list companies (cached)
/api/admin/companies/{id}/approve	POST	Approve a company
/api/admin/students	GET	Search/list students (cached)
/api/admin/drives	GET	List/filter placement drives
/api/admin/stats	GET	Funnel, trend & skills-demand analytics (cached)
/api/company/dashboard	GET	Company profile, drives, status breakdown
/api/company/drives	POST	Create a placement drive
/api/company/drives/{id}	GET	View a drive with applicants & interviews
/api/company/applications/{id}/status	POST	Update status (auto-creates Placement, queues offer letter)
/api/company/drives/{id}/interviews	POST	Schedule an interview
/api/student/dashboard	GET	Student profile, applications, status breakdown
/api/student/drives	GET	Browse/search open drives with eligibility flags

YAML API Definition File: Included separately in the submission ZIP as api.yaml.

7. Architecture and Features

Architecture Overview:

- run.py — entry point; creates the Flask app and the Celery instance
- backend/app.py — application factory; registers blueprints; serves the Vue SPA
- backend/extensions.py — db/bcrypt singletons (avoids circular imports)
- backend/models/ — SQLAlchemy models (7 tables)
- backend/routes/ — Flask blueprints: auth, admin, company, student, public
- backend/auth.py — JWT creation/verification, role-based decorators
- backend/cache.py — Redis cache wrapper with graceful fallback
- backend/celery_app.py + backend/jobs/tasks.py — Celery config and the four background jobs
- frontend/ — Vue 3 SPA (CDN, no build step): api.js, store.js, validate.js, components.js, page components, router

Implemented Features:

- JWT-based authentication with role-based access control (Admin / Company / Student)
- Company registration gated behind admin approval; drive creation gated behind company approval
- Admin: dashboard, company/drive approval, search (cached), blacklist/restore, analytics with Chart.js
- Company: drive management, applicant review, status updates, interview scheduling
- Student: drive browsing/search with eligibility checks, apply flow (duplicate- and deadline-checked), profile & resume management
- Complete application/placement history with role-scoped visibility (admin sees all, company sees own, student sees own)
- Redis caching on search/listing endpoints with active invalidation on writes
- Three scheduled/triggered Celery jobs plus an offer-letter generation job, all emailing via SMTP
- Client-side form validation with inline error messages, backed by server-side validation on every write endpoint

Additional Features:

- Auto-generated placement confirmation / offer letter, emailed and downloadable by the student
- Pre-login public landing page with cached aggregate placement statistics
- IDOR-safe file access — resumes and offer letters are ownership-checked, not just role-checked

8. Video Presentation

Drive Link: <https://drive.google.com/file/d/1x9h2mS4wdlQe-cq4g7j9kX7M8pXES-Wc/view?usp=sharing>